

Datos Generales

- Grupo base: Grupo XX
- Integrante: Martinez Steeven
- Proyecto: Sistema de Gestion de Caja de Ahorros
- Repositorio: <https://github.com/steevenmc04/Proyecto-Ing-Software>

Objetivo

Crear una aplicacion backend funcional utilizando herramientas de Ingenieria de Software, aplicando arquitectura por capas, servicios REST, documentacion Swagger y control de versiones con GitHub.

Framework Utilizado

FastAPI.

Servicios Publicados

- Swagger: <http://127.0.0.1:8000/docs>
- Redoc: <http://127.0.0.1:8000/redoc>
- Pagina funcional: <http://127.0.0.1:8000>

Arquitectura

El backend utiliza una estructura modelo, repositorio, servicio, controlador y ruta. Esta organizacion separa responsabilidades y permite mantener reglas de negocio independientes de la capa HTTP.

Evidencias De Cumplimiento

- Repositorio documentado con README y carpeta docs.
- Backend funcional con FastAPI.
- Servicios documentados con Swagger.
- Base de datos relacional mediante SQLAlchemy.
- Soporte para SQLite y MySQL Workbench.
- Pruebas basicas con pytest.
- Pagina funcional con login, menu y control por rol.
- API externa protegida con X-API-KEY.

Conclusiones

El desarrollo de la Tarea T02.03 permitio transformar los requerimientos y el diseno del Sistema de Gestion de Caja de Ahorros en una aplicacion backend funcional. Durante la implementacion se aplico una arquitectura por capas que separa modelos, repositorios, servicios, controladores y rutas, lo que facilita el mantenimiento y la comprension del codigo. FastAPI resulto adecuado para publicar servicios REST documentados con Swagger, mientras que SQLAlchemy permitio representar relaciones reales entre usuarios, socios, cuentas, transacciones, aportaciones, creditos, cuotas y asientos contables.

La construccion tambien evidencio la importancia de validar reglas de negocio desde el backend, especialmente en operaciones financieras como retiros, aportaciones, creditos y pagos. Adicionalmente, la incorporacion de autenticacion JWT y restricciones por rol permite que los socios consulten solo su informacion asociada, mejorando la seguridad del sistema. Las pruebas automatizadas con pytest ayudan a verificar que los flujos principales funcionen correctamente y reducen el riesgo de errores al modificar el proyecto.

Finalmente, el uso de GitHub como repositorio permite evidenciar el avance del trabajo, documentar la solucion y mantener un historial de cambios organizado para la entrega academica. Esta experiencia consolido buenas practicas de documentacion, verificacion continua y despliegue local reproducible para futuras iteraciones del sistema academico.